

LYCEUM OF THE PHILIPPINES UNIVERSITY MANILA, PHILIPPINES

**COLLEGE OF TECHNOLOGY
BACHELOR OF SCIENCE IN COMPUTER SCIENCE
2ND SEMESTER AY 2025-2026**

**(CAON32C)
Computer Architecture and Organization**

COURSE PORTFOLIO



Name:	Cruz, Justin Andrei D.
Member (s):	
Section:	C331_CS
Course Instructor / Professor:	Prof. Lazaro Josefina A.

TABLE OF CONTENTS	
	Page
Course Syllabus	3
Course Materials / References	9
Assessment Task (Week 1)	15
Assessment Task (Week 2)	19
Assessment Task (Week 3)	20
Assessment Task (Week 4)	21
Assessment Task (Week 5)	22
Prelim Examination	23
Assessment Task (Week 7)	24
Assessment Task (Week 8)	26
Assessment Task (Week 9)	27
Assessment Task (Week 10)	28
Assessment Task (Week 11)	
Midterm Examination	
Assessment Task (Week 13)	
Assessment Task (Week 14)	
Assessment Task (Week 15)	
Assessment Task (Week 16)	
Assessment Task (Week 17)	
Final Examination	
Final Project	
Rubrics	

COURSE SYLLABUS

PRELIM				
Lesson Learning Outcomes (LLOs)	Learning Content	Learning Strategies/ Methodologies	Learning Assessment Tools	Weeks
- Discuss all the necessary requirements such as attendance, seatwork, written quizzes, projects, and compliance of standard policy. - Discuss the syllabus, course outline and book references - Demonstrate an understanding of computer architecture by explaining its role and importance, and accurately converting numbers between binary, decimal, octal, and hexadecimal systems	Class Orientation: - Requirements and Grading system - Internalize the University & - College Mission, Vision and Core values - Academic Guidelines - House Rules & Seat Plan Introduction to Computer Architecture and Number Systems - Overview of Computer Architecture and Organization - Number Systems: Binary, Decimal, Hexadecimal, and Octal	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities - DOS BOX	1
Compare low-level and high-level programming languages, explain the structure and syntax of machine language, and develop simple machine-language	Programming Levels and Machine Language - Low-Level vs. High-Level Programming - Characteristics of Machine Language - Introduction to assembly language	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities	2

Analyze the historical development of microprocessors, measure the architectural advancements of the 8086 microprocessor, and evaluate the impact of microprocessor evolution on modern computing	Evolution of Microprocessors - History of microprocessor development - Generations of microprocessors - Overview of the 8086-microprocessor family	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities - DOS BOX	3
Develop a comprehensive explanation of memory segmentation and its role in program execution, demonstrate the use of internal registers in memory operations with clear examples, and design a memory map for simple applications with labeled sections and defined memory addresses	Memory Allocation and Internal Registers - Memory map creation and usage - Memory segmentation techniques - Functions of internal registers	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities - DOS BOX	4
Develop and debug simple assembly language programs	Assembly Language Formats	- Lecture - Visual	- Assignments - Quizzes	5
Grades Consultation and Completion of Requirements Prelim Examination / Submission of Major Requirements				6

MIDTERM				
Lesson Learning Outcomes (LLOs)	Learning Content	Learning Strategies/ Methodologies	Learning Assessment Tools	Weeks
Apply assembler directives to efficiently organize memory, debug segmentation-related issues, and clearly explain their role in structuring assembly programs using measurable outputs such as memory utilization rates, issue resolution time, and clarity of program structure.	Assembler Directives - Simplified segmentation directives - Standard segmentation directives	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities - DOS BOX	7
Use MOV instructions to transfer data between registers and memory, INT instructions to execute system-level operations, and labels to	Basic Instruction Sets - MOV instructions for data transfer - INT instructions for system calls - Labels for flow control	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities - DOS BOX	8

Use MOV instructions to transfer data between registers and memory, INT instructions to execute system-level operations, and labels to control program flow effectively, ensuring efficient data handling, precise system calls, and logical execution paths	Basic Instruction Sets - MOV instructions for data transfer - INT instructions for system calls - Labels for flow control	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities - DOS BOX	8
Develop assembly programs that utilize conditional branching for decision-making, implement loops for repetitive tasks, and debug branching logic errors to ensure accurate program execution.	Branching and Looping Instructions - Conditional and unconditional jumps - Looping instructions for iterative processes	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities - DOS BOX	9

Demonstrate proficiency in assembly language by performing arithmetic operations, using logical instructions for data manipulation, and solving practical problems involving bitwise operations; identify and explain various addressing modes, apply them to optimize program memory usage, and debug errors related to incorrect addressing.	Arithmetic and Logical Instructions - Addition, subtraction, multiplication, and division - Logical operations: AND, OR, NOT, XOR Addressing Modes - Register addressing - Immediate addressing - Direct and indirect addressing	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation - Collaborative Activities - DOS BOX	10
Develop reusable procedures and write macros to simplify repetitive operations, then	Procedures and Macros - Defining and calling procedures - Using macros for reusable code	- Lecture - Visual Presentation - Interactive Discussion	- Assignments - Quizzes - Student Participation	11
Grades Consultation and Completion of Requirements Midterm Examinations / Submission of Major Requirements				12

COURSE MATERIALS (LECTURES / REFERENCES)

LPU College of Technology 2nd Semester, AY 25-26 Computer Architecture and Organization (CAON32C) OVERVIEW OF COMPUTER ARCHITECTURE AND ORGANIZATION Ms. Josefina A. Lazaro, LPT	Learning Objectives: • Differentiate computer architecture and organization. • Identify major computer components and their functions. • Explain how instructions are executed by the CPU. • Describe the memory hierarchy and basic I/O methods.
Overview Computer Architecture vs Computer Organization Computer Architecture What to do? • Computer Architecture refers to how hardware components connect to form a computer system. • It's a design blueprint & decided first. • It acts as an interface between hardware and software. Computer Organizations How to do? • Computer Organization refers to the structure and behavior of the computer system as seen by the user. • It's decided after computer architecture. • It deals with the interconnection of components.	What is meant by Computer Architecture? • Computer architecture refers to the design and basic structure of a computer system, including its hardware structures, their interconnections, and the principles that guide their organization. • It encompasses the higher-level aspects of computer design, such as the instruction set architecture (ISA), memory hierarchy, and the organization of the central processing unit (CPU). • Computer architecture sets the foundation for building efficient, reliable, and high-performance computer systems.
Basic Architecture of a Computer	The 3 main categories are considered while considering the design of architecture: 1. System Design – contains hardware components that are used for building the system. 2. Instruction Set Architecture (ISA) – includes all the instructions provided to the computer system. 3. Micro Architecture – give minute detail about the storage element
What is meant by Computer Organization? • Computer organization describes how a computer's hardware components are connected and work together to execute tasks. • It involves the design and arrangement of physical components like the CPU, memory, I/O devices, and control units for efficient operation. • It focuses on the computer's functional units and how they interconnect to implement the architectural design.	Types of CPU Organization • Single Accumulator Organization • General Register Organization • Stack Organization



College of Technology
2nd Semester, AY 25-26

Computer Architecture and Organization
(CAON32C)

NUMBER SYSTEM
(Binary, Decimal, Octal, Hexadecimal)

Ms. Josefina A. Lazaro, LPT





Learning Objectives:

- Understand the concept of number systems.
- Identify the four major number systems used in computing.
- Convert numbers from one base to another.
- Perform basic arithmetic in binary.

"Number systems are not just math. They are the foundation of how data is stored, transmitted, and processed in all computer systems."

What is the NUMBER SYSTEM?

Number System is a method of representing numbers with the help of a set of symbols and rules; it is a mathematical notation used to represent quantities or values in various forms.

A number system, or numeral system, is a method for representing numbers using symbols, crucial in mathematics and programming for data representation.

All data in computers is processed as binary code.

Examples: .jpg File, .mp3 File, .txt File

- A number system uses a base (or radix) to represent values.
- The base refers to the number of unique digits, including zero, that a system uses to represent numbers.
- Each system has its own set of rules for representing.

Types of Number Systems

Number System

- Decimal Number System**
Numbers with base 10
EX: $(102)_{10}, (24.57)_{10}, \dots$
- Binary Number System**
Numbers with base 2
EX: $(1001)_{2}, (101.000)_{2}, \dots$
- Octal Number System**
Numbers with base 8
EX: $(240)_{8}, (12.57)_{8}, \dots$
- Hexadecimal Number System**
Numbers with base 16
EX: $(AB)_{16}, (0.3F)_{16}, \dots$

Types of Number Systems

The **Decimal Number System** is the standard system for denoting numbers.

- It is also called the **base-10** system.
- Digits used in it are **0, 1, 2, 3, 4, 5, 6, 7, 8, 9**.
- Each digit in the number is at a specific place value that is a power of 10.
- From right to left - units have the place value as 10^0 , Tens have the place value as 10^1 , Hundreds have the place value as 10^2 , Thousands have the place value as 10^3 , and so on..

Example:

10285 can be written as

$$10285 = (1 \times 10^4) + (0 \times 10^3) + (2 \times 10^2) + (8 \times 10^1) + (5 \times 10^0)$$

$$10285 = 1 \times 10000 + 0 \times 1000 + 2 \times 100 + 8 \times 10 + 5 \times 1$$

$$10285 = 10000 + 0 + 200 + 80 + 5$$

$$10285 = 10285$$

Types of Number Systems

The **Binary Number System** is the number system with **base 2**.

- The numbers are formed using two digits - **0 and 1**.
- The binary number system is beneficial in electronic devices and computer systems because it can be easily performed using just two states, i.e., 0 and 1.
- Each digit in the number is at a specific place value that is a power of 2.
- From right to left - as powers of 2 i.e. $2^0, 2^1, 2^2$, etc).

Example:

$(1011)_2$ can be written as

$$(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$(1011)_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$$

$$(1011)_2 = 11 \text{ (in decimal)}$$

Types of Number Systems

The **Octal Number System** is the number system with **base 8**.

- The numbers are formed using 8 digits i.e. **0, 1, 2, 3, 4, 5, 6, 7**.
- The octal number system helps represent file permissions in Unix/Linux operating systems.
- Each digit in the number is at a specific place value that is a power of 8.
- From right to left - as powers of 8 i.e. $8^0, 8^1, 8^2$, etc.

Example:

$(325)_8$ can be written as

$$(325)_8 = 3 \times 8^2 + 2 \times 8^1 + 5 \times 8^0$$

$$(325)_8 = 192 + 16 + 5$$

$$(325)_8 = 213 \text{ (in decimal)}$$

Types of Number Systems

The **Hexadecimal Number System** is the number system with **base 16**.

- The numbers are formed using 16 digits, i.e., 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.
- Hexadecimal Numbers help handle memory address locations.
- The digits from **0 to 9** are used as in the decimal system, but the numbers **10 to 15** are represented using the letters **A to F** as follows: 10 is described as **A**, 11 as **B**, 12 as **C**, 13 as **D**, 14 as **E**, 15 as **F**.
- Each digit in the number occupies a specific place value, which is a power of 16.
- From right to left - as powers of 16 i.e. $16^0, 16^1, 16^2$, etc.

Example:

$(2F)_{16}$ can be written as

$$(2F)_{16} = 2 \times 16^1 + F \times 16^0$$

$$(2F)_{16} = 2 \times 16 + 15 \times 1$$

$$(2F)_{16} = 32 + 15$$

$$(2F)_{16} = 47 \text{ (in decimal)}$$

ASSEMBLER DIRECTIVES

Simplified & Standard Segmentation Directives

.MODEL .STACK .DATA .CODE

SEGMENT ENDS ASSUME

```
.MODEL SMALL
.STACK 100h
.DATA
msg DB 'Hello!'
CODE
MAIN PROC
MOV AX,@DATA
...
```

Lesson Overview

- 01 What Are Assembler Directives?
Commands for the assembler, not the processor
- 02 Simplified Segmentation Directives
.MODEL .STACK .DATA .CODE .EXIT
- 03 Standard Segmentation Directives
SEGMENT, ENDS, ASSUME, GROUP
- 04 Code Examples & Comparison
Side-by-side syntax and usage
- 05 Summary
Key takeaways

What Are Assembler Directives?

Definition

Assembler directives (also called pseudo-ops or pseudo-instructions) are commands that guide the assembler during the translation of source code into machine code.

They are NOT executable instructions; they do not produce machine code themselves.

Key Characteristics

- Processed at assembly time, not runtime
- Tell the assembler how to handle code
- Define memory segments and structure
- Allocate storage for variables
- Define constants and equates
- Control listing and linking options

FUNDAMENTALS

What Are Assembler Directives?

Definition: Assembler directives are instructions to the assembler program itself — not to the CPU. They guide the assembly process but do NOT generate executable machine code.

Assembler Directives

Tell the assembler HOW to process the code

Do not produce machine code bytes (mostly)

Examples: DB, SEGMENT, EQU, MACRO

Active at assembly time

Machine Instructions

Tell the CPU WHAT to do at runtime

Always produce machine code bytes

Example: MOV, ADD, INT, JMP

Active at execution time

Simplified Segmentation Directives: Introduction

Simplified segmentation directives were introduced in MASM 5.0 to reduce the amount of boilerplate code required to define memory segments. They automatically set up the standard segments (Code, Data, Stack) using simple, readable keywords.

Easier Syntax

No need to manually define SEGMENT/ENDS blocks. Use shorthand directives like .DATA and .CODE.

Less Code

Automatically handles segment names, class attributes, and ASSUME, reducing repetitive setup.

Requires .MODEL

Must declare a memory model using .MODEL directive before using any simplified segment directives.

Simplified Segmentation: .MODEL Directive

Syntax: .MODEL memory-model [, language-type] [, stack-option]

Model	Code Segments	Data Segments	Max Code	Max Data	Use Case
TINY	1 (shared)	1 (shared)	64 KB total	64 KB total	.COM programs
SMALL	1	1	64 KB	64 KB	Small DOS programs ★
MEDIUM	Multiple	1	> 64 KB	64 KB	Medium programs
COMPACT	1	Multiple	64 KB	> 64 KB	Data-heavy programs
LARGE	Multiple	Multiple	> 64 KB	> 64 KB	Large programs
HUGE	Multiple	Multiple	> 64 KB	> 64 KB each	Very large arrays

Simplified Segmentation Directives — Directive List

.STACK [size]	Defines the Stack Segment. Optional size in bytes (default: 1024). Common: .STACK 100h (256 bytes).
.DATA	Defines the initialized Data Segment. Variables with initial values are declared here.
.DATA?	Defines the uninitialized Data Segment. Variables with storage but no initial value.
.CONST	Defines a read-only Constant Data Segment. Constants that should not be modified.
.CODE [name]	Defines the Code Segment. All executable instructions go here. Optional name parameter.
.FARDATA [name]	Defines a far Data Segment for use with COMPACT, LARGE, or HUGE memory models.
.FARDATA? [name]	Defines a far uninitialized Data Segment for COMPACT, LARGE, or HUGE models.

Simplified Segmentation — Complete Example

Program Code

```
; Simplified Segmentation Example
.MODEL SMALL
.STACK 100h

.DATA
msg DB 'Hello, World!'
num DB 25

.CODE
MAIN PROC
MOV AX, @DATA
MOV DS, AX
LEA DX, msg
MOV AH, 09h
INT 21h
.EXIT
MAIN ENDP
END MAIN
```

Annotations

.MODEL SMALL
Sets the memory model. Code and data each fit within 64KB.

.STACK 100h
Reserves 256 bytes (0x100) for the stack segment.

.DATA
Opens the data segment. Declare all initialized variables here.

@DATA
Symbolic constant — address of the data segment.

.EXIT
Macro that calls INT 21h/4Ch — cleanly exits to DOS.

END MAIN
Marks end of source file; MAIN is the entry point.



Assembly Language Essentials: Instruction Sets, Control Flow, Addressing, Procedures & Macros

Focused on x86 (8086/real-mode style) syntax examples

Suitable for MASM/TASM/NASM (syntax notes included)

Learn Different. Live Different.



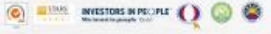
Learning Goals & Agenda

- Basic instruction sets
 - MOV (data transfer)
 - INT (system calls / interrupts)
 - Labels (flow control)
- Branching & looping
 - Conditional & unconditional jumps
 - LOOP-family instructions
- Arithmetic & logical ops
 - Addition/subtraction/multiplication/division
 - AND/OR/NOT/XOR + TEST
- Addressing modes
 - Register, immediately
 - Direct, indirect, indexed/based
- Procedures & macros
 - Define/call procedures
 - Use macros for reusable code



Part 1 — Basic Instruction Sets

Core building blocks:
moving data, calling services, and structuring flow





MOV: Data Transfer (Register ↔ Memory ↔ Immediate)

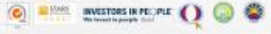
- Copies data from source to destination (typically does NOT change FLAGS).
- Common forms
 - MOV reg, imm
 - MOV reg, reg
 - MOV reg, [mem]
 - MOV [mem], reg
- Rules
 - No memory-to-memory MOV (use a register).
 - Operand sizes must match (BYTE PTR/WORD PTR when needed).
- Used for initialization, passing values, and load/store operations.

```

; MASM/TASM-style examples
mov ax, 1234h      ; immediate → register
mov dx, ax          ; register → register
mov ax, [bx]        ; memory → register
mov [di], ax        ; register → memory

; also override examples
mov byte ptr [di], 0FFh
mov word ptr [bx], 002Ah

```





INT: Software Interrupts / System Calls

- INT n triggers interrupt vector n and transfers control to an interrupt handler.
- In real mode, used for BIOS/DOS services (e.g., INT 10h video, INT 21h DOS API).
- Typical pattern
 - Load function number + parameters into registers
 - Execute INT
 - Read results from registers/lags
- Modern OSes use different syscall mechanisms, but the concept is similar.

```

; Print character 'A' via DOS (real mode)
mov ah, 02h
mov dl, 'A'
int 21h

; Int video mode 80x via BIOS
mov ah, 00h
mov al, 03h
int 10h

```





Labels: Naming Locations for Flow Control

- Labels mark addresses in code (or data) so jumps/calls can target them.
- Why they matter
 - Readable flow
 - Enable loops/branches
 - Support structured patterns
- Good practices
 - Meaningful names
 - Avoid excessive jumping
 - Keep scope small

```

start:
mov cx, 5
loop_start:
; ... do work ...
dec cx
jnz loop_start
done:
; continue

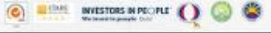
```





Part 2 — Branching and Looping

Changing normal sequential execution





JMP & Jcc: Unconditional vs Conditional Jumps

- Unconditional
 - JMP label — always jumps
- Conditional (Jcc)
 - Based on FLAGS set by CMP/TEST/arithmetic
 - JE/JZ, JNE/JNZ, JL/JG (signed), JB/JA (unsigned)
- Workflow
 - Set FLAGS (CMP/TEST/ADD/SUB)
 - Choose the correct Jcc
- Always choose signed vs unsigned jumps correctly.

```

; if (a == b) do1 else do=0
cmp ax, bx
je equal
mov dx, 0
jmp done
equal:
mov dx, 1
done:

```



ASSESSMENT TASK (Week 1)

Evolution of Microprocessors

I. Timeline Mapping (25-points)

Instructions:

Create a concise timeline showing the evolution of microprocessors from the **Intel 4004** (1971) up to modern multicore processors. Your timeline must include:

- At least 5 major milestones
- A short description of each (2–3 sentences)
- One key improvement introduced per generation

Guide Questions:

1. What trend do you notice in terms of processor speed and functionality?
2. How did microprocessor evolution influence modern computing devices?

Rubric Scoring

Criteria
Accuracy of Content (5 pts)
Descriptions of Milestones (5 pts)
Identification of Key Improvements (5 pts)
Quality of Insights in Guide Questions (10 pts)

II. Generations Comparison Table (20-points)

A. Fill in the table with the correct information about microprocessor generations.

Generation	Example microprocessor	Year Introduced	Key Characteristics	Importance
1st Gen				
2nd Gen				
3rd Gen				
4th Gen				
5th Gen				

Rubric Scoring

A. Comparison Table (10 points)

Criteria
Completeness of Table (5 pts)
Accuracy of Characteristics & Importance (5 pts)

- B. **Task:** After completing the table, write a 2-3 paragraph (not sentences) comparing **3rd** and **5th** generation microprocessors. Focus on improvements in architecture and performance.

B. Comparison Paragraph (10 points)

Criteria
Depth of Comparison (4 pts)
Clarity & Organization (3 pts)
Technical Accuracy (3 pts)

III. 8086 Family Quick Analysis (30-points)

Research or recall information about the **8086-microprocessor family** and answer:

1. What addressing mode innovations were introduced by the 8086?
2. Why is the 8086 considered a “foundation” of modern x86 processors?
3. List and briefly describe **two** strengths and **two** limitations of the 8086 architectures.

Rubric Scoring

Criteria
Addressing Mode Innovations (10 pts)
Reason for Foundation of x86 (10 pts)
Strengths & Limitations (10 pts)

Overall Total Points: 75

I. Timeline Mapping

1. Intel 4004 (1971) – First Commercial Microprocessor

The Intel 4004 was the world's first commercially available microprocessor, originally designed for calculators. It was a 4-bit processor capable of performing basic arithmetic and logical operations. Although limited in speed and memory capacity, it marked the beginning of integrating a CPU into a single chip.

Key Improvement: Introduction of a single-chip CPU architecture.

References:

Intel Corporation. (n.d.). *Intel 4004 microprocessor*. Intel Museum. <https://www.intel.com/content/www/us/en/history/museum-story-of-intel-4004.html>

2. Intel 80386 (1985) – 32-bit Processing Era

The Intel 80386 (386) expanded computing power with a full 32-bit architecture. It supported multitasking and virtual memory, which significantly improved operating system performance. This generation made personal computers more powerful and business-ready.

Key Improvement: 32-bit architecture with virtual memory support.

References:

Intel Corporation. (n.d.). *Intel 80386 processor*. <https://www.intel.com/content/www/us/en/history/museum-story-of-intel-386.html>

3. Intel Pentium (1993) – Superscalar Architecture

The Pentium processor introduced superscalar architecture, allowing it to execute multiple instructions per clock cycle. It significantly increased speed and multimedia capabilities. This generation improved floating-point performance, benefiting graphics and gaming applications.

Key Improvement: Superscalar processing (parallel instruction execution).

References:

Intel Corporation. (n.d.). *Intel Pentium processor family*. <https://www.intel.com/content/www/us/en/history/museum-story-of-pentium.html>

4. Intel Core i7 (2008) – Multi-Core and Hyper-Threading

The Intel Core i7 (Nehalem architecture) marked the shift toward multi-core processors becoming mainstream. It integrated memory controllers and supported

Hyper-Threading, allowing better parallel processing. This improved efficiency, multitasking, and performance for demanding applications.

Key Improvement: Multi-core processing with integrated memory controller.

References:

Intel Corporation. (2008). *Intel Core i7 processor overview*. <https://www.intel.com>

5. Apple M1 (2020) – System on Chip (SoC) & Energy Efficiency

The Apple M1 represents modern microprocessor design with ARM-based architecture and system-on-chip integration. It combines CPU, GPU, and memory into a single chip for high efficiency and performance. It demonstrates the shift toward performance-per-watt optimization in modern devices.

Key Improvement: Unified SoC design with high energy efficiency.

References:

Apple Inc. (2020). *Apple unveils M1 chip*. Apple Newsroom. <https://www.apple.com/newsroom/2020/11/apple-unveils-m1/>

Guide Questions

1. I think the most noticeable trend is that processors became faster, smaller, and more powerful over time. In the early generations like the Intel 4004, processors were very limited and could only handle simple computations. As generations progressed, especially from the 80386 to Pentium and modern Core processors, the focus shifted not just to increasing clock speed but also to improving architecture, adding more cores, and supporting multitasking.

I also noticed that modern processors prioritize efficiency and parallel processing instead of just raw speed. Instead of increasing GHz aggressively, manufacturers improved multi-core designs and power management. This shows that performance today is more about smart design and workload distribution rather than just higher frequency.

2. I believe microprocessor evolution is the main reason why we now have powerful laptops, smartphones, and even cloud servers. If processors did not evolve from 4-bit to 64-bit multi-core systems, we would not be able to run modern operating systems, AI applications, or high-end games. Every improvement in architecture made devices more capable and more accessible to people.

In my opinion, the shift to multi-core and system-on-chip designs also made devices thinner, more energy-efficient, and portable. For example, modern chips like Apple's M1 combine CPU, GPU, and memory into one system, which improves performance while reducing power consumption. Because of these advancements, computing is no longer limited to desktops and it is now embedded in phones, smart devices, and even IoT systems.

II. Generation Comparison Table

Generation	Example Microprocessor	Year Introduced	Key Characteristics	Importance
1 st Gen	Intel 4004	1971	4-bit processing, basic instruction set, limited memory	First single-chip microprocessor; foundation of modern CPUs
2 nd Gen	Intel 8086	1978	16-bit architecture, improved memory addressing, x86 instruction set	Established x86 standard used in modern computers
3 rd Gen	Intel 80386	1985	32-bit processing, virtual memory, multitasking support	Enabled advanced operating systems like GUI-based system
4 th Gen	Intel Pentium	1993	Superscalar design, improved	Boosted multimedia, gaming, and

			floating-point unit	business applications
5 th Gen	Intel Core i7	2008	Multi-core, Hyper-Threading, integrated memory controller	Supports high-performance computing and modern multitasking

B. Comparison Paragraph

- As a third-year student studying Computer Science in LPU Manila, I can observe that microprocessor evolution reflects Moore's Law in action and its transistors doubled while size decreased and performance increased. The shift from single core to multi-core processors shows that innovation is no longer just about speed, but about efficiency, parallelism, and intelligent workload distribution. These advancements directly influence fields like cybersecurity, software engineering, AI development, and cloud computing, which rely heavily on processor capability.

III. 8086 Family Quick Analysis

1.

- The 8086 introduced several advanced addressing modes compared to earlier processors like the Intel 4004 and 8080. I think one of the biggest innovations was **segmented memory addressing**. The 8086 used segment registers (CS, DS, SS, ES) combined with 16-bit offsets to access up to 1MB of memory, even though it was only a 16-bit processor. This was innovative because it extended memory access beyond the 64KB limit of earlier CPUs.

Another major innovation was the variety of **flexible addressing modes**, including:

- Immediate addressing
- Register addressing
- Direct memory addressing
- Register indirect addressing
- Based and indexed addressing
- Based-index with displacement

I believe these modes made programming more efficient because they allowed complex data structures like arrays and stacks to be accessed more easily. This improved performance and gave programmers more flexibility in writing assembly programs.

2. The 8086 is considered the foundation of modern x86 processors because it introduced the **x86 instruction set architecture (ISA)**. Modern Intel and even AMD processors still maintain backward compatibility with this original instruction set.

In my opinion, what makes it foundational is that future processors like the 80286, 80386, Pentium, and even modern Core series extended the same architecture instead of replacing it. This means software written decades ago can still run on modern CPUs because of backward compatibility.

I think this design decision shaped the entire PC industry. When IBM selected an 8086-based processor for early PCs, it standardized the x86 platform. Because of that, x86 became dominant in desktops and laptops, influencing operating systems like Windows and Linux.

3.

Strengths

1. Expanded Memory Access (1MB Address Space)

The segmented memory model allowed access to 1MB of RAM, which was a big improvement compared to previous 8-bit processors limited to 64KB. I think this made the 8086 suitable for more advanced applications and operating systems.

2. Rich Instruction Set

The 8086 had a relatively powerful and flexible instruction set. It supported string operations, arithmetic, logical operations, and stack management. This made it more versatile for software development.

Limitations

1. Segmented Memory Complexity

Although segmentation expanded memory, it also made programming more complicated. I think managing segment registers increased the chances of programming errors and made memory handling less straightforward.

2. Lack of Built-in Floating-Point Unit (FPU)

The 8086 did not have an integrated floating-point unit. Complex

mathematical operations required a separate coprocessor (like the 8087). This limited performance in scientific and engineering applications.

References:

GeeksforGeeks. (2025, September 12). *Addressing modes in 8086 microprocessor*.

<https://www.geeksforgeeks.org/computer-organization-architecture/memory-segmentation-8086-microprocessor/>

GeeksforGeeks. (2025, September 12). *Architecture of 8086*.

<https://www.geeksforgeeks.org/computer-organization-architecture/architecture-of-8086/>

GeeksforGeeks. (2025, September 12). *Addressing modes in 8086*.

<https://www.geeksforgeeks.org/addressing-modes/>

Tpoint Tech. (n.d.). *8086 microprocessor: Addressing modes*.

<https://www.tpointtech.com/8086-microprocessor>

General History & x86 Foundation

Wikipedia contributors. (2026, February). *Intel 8086*. Wikipedia.

https://en.wikipedia.org/wiki/Intel_8086

Wasil Zafar. (2026). *x86 Assembly Series Part 2: x86 CPU architecture overview*.

<https://www.wasilzafar.com/pages/series/assembly-mastery/asm-part02-cpu-architecture.html>

Research Activity 1 - Evolution of Microprocessors

Due February 24, 2026 4:00 PM • Closes February 24, 2026 4:00 PM

Points

75 / 75

Instructions

None

Reference materials



Research Activity.docx



My work



CRUZ_RA1.pdf



QUIZ 1

Respondent: **CRUZ JUSTIN ANDREI**

Last name: CRUZ

First name: JUSTIN ANDREI

Start date: 2026-02-20 15:07

Score: **(22/40)**

Essay

SHORT CASE STUDY

NAME: JUSTIN ANDREI D. C371-CS

NO.:
DATE: Feb 3, 2016

1. IDENTIFY THREE POSSIBLE COMPONENTS INVOLVE CAUSING THE ISSUE?

I THINK THE THREE POSSIBLE COMPONENTS WHY THE STUDENT'S LAPTOP ARE VERY SLOW. THE FIRST ONE IS MAYBE THE LAPTOP USES HDD (HARD DISK DRIVE) STORAGE DEVICES, HDD STORAGE IS AN OLD SCHOOL. THE NEXT ONE IS RAM (RANDOM ACCESS MEMORY), THE LAST ONE IS MAYBE THE I/O PORTS MIGHT BE MALFUNCTIONING.

2. Explain THE CAUSE.

IF THE LAPTOP USES A NORMAL HDD AND NOT SSD. THE PHYSICAL "READ/WRITE" HEADS HAVE TO MOVE TO FIND DATA. HDD IS SLOWER THAN SSD (SOLID STATE DRIVE). WHEN RAM IS FULL, THE OS USES "VIRTUAL MEMORY" ON THE SLOW HARD DRIVE. THIS CAUSES MASSIVE PERFORMANCE DROP. THE NEXT ONE IS I/O THROUGHPUT LIKE USB 2.0 AND 3.0. 3.0 IS FASTER THAN 2.0.

3. PROPOSE PRACTICAL SOLUTIONS/RECOMMEND 3 CONCRETE WAYS TO IMPROVE PERFORMANCE AND EXPLAIN WHY EACH SOLUTIONS WORK.

I THINK THE MOST PRACTICAL SOLUTION TO MAKE THE LAPTOP FASTER IS IF ITS UPGRADEABLE RAM, UPGRADE THE RAM TO THE LAPTOP'S MAXIMUM CAN HANDLE WITH THAT, IT HAS A LOT OF STORAGE SO THE OS WILL NOT USE THE VIRTUAL RAM. 2nd ONE IF THE 1st AVAILABLE SOLUTIONS ARE NOT WORKING, DELETE UNNECESSARY DATA OR APPS, IT IS THE GREAT WAY TO REDUCE STORAGE USAGE AND MAKE THE LAPTOP FASTER. LASTLY, IF THE STUDENT HAVE A BUDGET, HE/SHE SHOULD BUY A LAPTOP THAT HAS 16 GB OF RAM AND USE SSD AS STORAGE.

QUIZ 2 – 74TH FOUNDATION

Q2-74th Foundation

Due March 4, 2026 11:59 PM

Instructions

Take an individual selfie of the event and upload it here. No selfie, no quiz2

My work

 IMG_20260304_094735.jpg ...

 IMG_20260304_094457.jpg ...

 [Attach](#)  [New](#)

DAY 3 – 74TH FOUNDATION ANNIVERSARY

Day3 - 74th Foundation Anniversary

Due March 7, 2026 11:59 PM

Points

No points

Instructions

Upload the proof of selfie from today's event, "Himigsikan," which start 4 PM at LPU Quad.

My work

IMG_20260306_163511 1.jpg

Attach + New



LPU
SYSTEM OF THE PHILIPPINES UNIVERSITY

PE

459171258

[illegible]

- | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|----|---|---|---|---|---|----|---|---|---|---|---|----|---|---|---|---|---|----|---|---|---|---|---|-----|---|---|---|---|---|
| 1 | A | B | C | D | E | 21 | A | B | C | D | E | 41 | A | B | C | D | E | 61 | A | B | C | D | E | 81 | A | B | C | D | E |
| 2 | A | B | C | D | E | 22 | A | B | C | D | E | 42 | A | B | C | D | E | 62 | A | B | C | D | E | 82 | A | B | C | D | E |
| 3 | A | B | C | D | E | 23 | A | B | C | D | E | 43 | A | B | C | D | E | 63 | A | B | C | D | E | 83 | A | B | C | D | E |
| 4 | A | B | C | D | E | 24 | A | B | C | D | E | 44 | A | B | C | D | E | 64 | A | B | C | D | E | 84 | A | B | C | D | E |
| 5 | A | B | C | D | E | 25 | A | B | C | D | E | 45 | A | B | C | D | E | 65 | A | B | C | D | E | 85 | A | B | C | D | E |
| 6 | A | B | C | D | E | 26 | A | B | C | D | E | 46 | A | B | C | D | E | 66 | A | B | C | D | E | 86 | A | B | C | D | E |
| 7 | A | B | C | D | E | 27 | A | B | C | D | E | 47 | A | B | C | D | E | 67 | A | B | C | D | E | 87 | A | B | C | D | E |
| 8 | A | B | C | D | E | 28 | A | B | C | D | E | 48 | A | B | C | D | E | 68 | A | B | C | D | E | 88 | A | B | C | D | E |
| 9 | A | B | C | D | E | 29 | A | B | C | D | E | 49 | A | B | C | D | E | 69 | A | B | C | D | E | 89 | A | B | C | D | E |
| 10 | A | B | C | D | E | 30 | A | B | C | D | E | 50 | A | B | C | D | E | 70 | A | B | C | D | E | 90 | A | B | C | D | E |
| 11 | A | B | C | D | E | 31 | A | B | C | D | E | 51 | A | B | C | D | E | 71 | A | B | C | D | E | 91 | A | B | C | D | E |
| 12 | A | B | C | D | E | 32 | A | B | C | D | E | 52 | A | B | C | D | E | 72 | A | B | C | D | E | 92 | A | B | C | D | E |
| 13 | A | B | C | D | E | 33 | A | B | C | D | E | 53 | A | B | C | D | E | 73 | A | B | C | D | E | 93 | A | B | C | D | E |
| 14 | A | B | C | D | E | 34 | A | B | C | D | E | 54 | A | B | C | D | E | 74 | A | B | C | D | E | 94 | A | B | C | D | E |
| 15 | A | B | C | D | E | 35 | A | B | C | D | E | 55 | A | B | C | D | E | 75 | A | B | C | D | E | 95 | A | B | C | D | E |
| 16 | A | B | C | D | E | 36 | A | B | C | D | E | 56 | A | B | C | D | E | 76 | A | B | C | D | E | 96 | A | B | C | D | E |
| 17 | A | B | C | D | E | 37 | A | B | C | D | E | 57 | A | B | C | D | E | 77 | A | B | C | D | E | 97 | A | B | C | D | E |
| 18 | A | B | C | D | E | 38 | A | B | C | D | E | 58 | A | B | C | D | E | 78 | A | B | C | D | E | 98 | A | B | C | D | E |
| 19 | A | B | C | D | E | 39 | A | B | C | D | E | 59 | A | B | C | D | E | 79 | A | B | C | D | E | 99 | A | B | C | D | E |
| 20 | A | B | C | D | E | 40 | A | B | C | D | E | 60 | A | B | C | D | E | 80 | A | B | C | D | E | 100 | A | B | C | D | E |

MIDTERMS ACTIVITY ASSEMBLY PROGRAMMING

Cruz, Justin Andrei D.

Computer Architecture

C331_CS

1.

```
section .data
msg db '1','2','3','4','5','6','7','8','9',10
len equ $ - msg

section .text
global _start

_start:
    mov eax, 4      ; sys_write
    mov ebx, 1      ; stdout
    mov ecx, msg
    mov edx, len
    int 0x80

    mov eax, 1      ; sys_exit
    int 0x80
```

STON

Input for the program (Optional)

Output: 123456789

2.

```
section .bss
result resb 4

section .text
global _start

_start:
    mov ecx, 5
    mov edx, 10

    call num        ; result in EAX

    ; convert number to ASCII (simple for 1-99)
    add eax, 48      ; convert to character
    mov [result], al

    ; print result
    mov ebx, 4
    mov ecx, 1
    mov edx, result
    mov esi, 1
    int 0x80

    ; exit
    mov ebx, 1
    int 0x80

num:
    mov eax, ecx
    add eax, ecx
    ret
```

STON

Input for the program (Optional)

Output: 100143100

3.

```
section .bss
char resb 1

section .text
global _start

_start:
    call display

    mov ecx, 1
    int 0x80

display:
    mov ecx, 0      ; counter

print_loop:
    mov [char], cl

    mov ebx, 4
    mov ecx, 1
    mov edx, 1
    int 0x80

    inc cl
    cmp cl, 255
    jne print_loop

    ret
```

STON

Input for the program (Optional)

Output: (4mm) 43 00

Output limit exceeded (max 65536 bytes)

4.

```
macro print_msg 2
    mov ebx, 4
    mov ecx, 1
    mov edx, %1
    mov ecx, %2
    mov edx, %3
    int 0x80
endmacro

section .data
msg1 db "Hello Programmers!", 10
len1 equ $ - msg1

msg2 db "Welcome to the world of,", 10
len2 equ $ - msg2

msg3 db "Linux Assembly programming!", 10
len3 equ $ - msg3

section .text
global _start

_start:
    print_msg msg1, len1
    print_msg msg2, len2
    print_msg msg3, len3

    mov ebx, 1
    int 0x80
```

STON

Input for the program (Optional)

Output: Hello Programmers!
Welcome to the world of,
Linux Assembly programming!

5.

<pre>section .data msg db 'Looping...', 10 len equ \$ - msg section .text global _start _start: mov ecx, 5 ; loop 5 times loop_start: mov eax, 4 mov ebx, 1 mov ecx, msg mov edx, len int 0x80 loop loop_start ; automatically decrements ECX mov eax, 1 int 0x80</pre>	STDIN Input for the program (Optional) Output: 7 ms 4.6 MB Looping... Looping... Looping... Looping... Looping... Looping... Looping... Looping... Looping... Looping... Looping... Looping...
---	--

MP QUIZ 1

Your final grade for this quiz is 32.00/40.00.

Your attempts

Attempt 1	
Status	Finished
Started	Friday, March 27, 2026, 2:52 PM
Completed	Friday, March 27, 2026, 3:12 PM
Duration	20 mins 8 secs
Grade	32.00 out of 40.00 (80%)
Review	

MP QUIZ 2

Your final grade for this quiz is 39.00/40.00.

Your attempts

Attempt 1	
Status	Finished
Started	Friday, April 10, 2026, 2:50 PM
Completed	Friday, April 10, 2026, 3:00 PM
Duration	10 mins 4 secs
Grade	39.00 out of 40.00 (97.5%)
Review	

CERT AI AMPLIFIED SCHOLAR



ASSESSMENT TASK (Week 10)

ASSESSMENT TASK (Week 11)

MAJOR EXAMINATION (MIDTERM)

ASSESSMENT TASK (Week 13)

ASSESSMENT TASK (Week 14)

ASSESSMENT TASK (Week 15)

ASSESSMENT TASK (Week 16)

ASSESSMENT TASK (Week 17)

MAJOR EXAMINATION (FINAL)

FINAL PROJECT

RUBRIC (SO 1 – 6)